Анализ данных с использованием языка Python

Пояснения к коду. Семинар 12

Гипотеза о среднем: одна выборка и доверительные интервалы

А. Н. Лата

Пояснения к ноутбуку `notebooks/seminar-12-one-sample.ipynb`. Номер ячейки — её порядковый номер среди ячеек кода ноутбука, считая сверху. Для каждой ячейки приведён её код с номерами строк, а под ним — что делает каждая команда и что означает каждый аргумент именно в этой ячейке. Команды, разобранные в пояснениях к семинарам 8–11, здесь только называются. Код совпадает с ноутбуком, только пробелы перед комментариями сокращены. Полные подписи функций со всеми параметрами — в справочнике `reference/statistics.md`, раздел «Одна выборка».

1 Вопрос, гипотеза, тест, вывод

Ячейка 1. Подключение и стаж водителей

```
1  import matplotlib.pyplot as plt
2  import numpy as np
3  import pandas as pd
4  import seaborn as sns
5  from scipy import stats
6  from statsmodels.stats.proportion import proportions_ztest, ↩
   ↪ proportion_confint
7
8  sns.set_theme(style='whitegrid')  # оформление графиков по умолчанию
9  dtp = pd.read_csv('../datasets/legal/dtp-bryansk.csv', index_col='id', ↩
   ↪ parse_dates=['datetime'])  # ДТП Брянской области
10 people = pd.read_csv('../datasets/legal/dtp-bryansk-participants.csv')  # ↩
   ↪ участники ДТП
11 people['experience_clean'] = people['experience'].replace([95, 96, 99], ↩
   ↪ np.nan)  # коды стажа пропусками
12 drivers = people[people['role'] == 'Водитель'].merge(dtp, ↩
   ↪ left_on='dtp_id', right_index=True)  # водители с их ДТП
13 drivers['vehicle_age'] = drivers['datetime'].dt.year - ↩
   ↪ drivers['vehicle_year']  # возраст машины в годах
14 drivers['vehicle_age'] = ↩
   ↪ drivers['vehicle_age'].where(drivers['vehicle_age'] <= 100)  # ↩
   ↪ невозможный возраст в пропуск
15 experience = drivers['experience_clean'].dropna()  # стаж без пропусков
16 print(len(experience), round(experience.mean(), 2), experience.median())
```

– `from statsmodels.stats.proportion import proportions_ztest, proportion_confint` — две функции для долей из statsmodels: тест и интервал. Через запятую можно подключить несколько функций сразу.

– Строки 8–15 — таблицы и стаж водителей без пропусков, как на семинаре 11.

Ячейка 2. t-тест одной выборки

```
result = stats.ttest_1samp(experience, 15)  # t-тест против 15
result
```

– `stats.ttest_1samp(experience, 15)` — t-тест одной выборки: первый аргумент — значения, второй — проверяемое среднее. По умолчанию альтернатива двусторонняя, «не равно».

– Результат — объект `TtestResult` с полями `statistic` — t, `pvalue` и `df` — число степеней свободы, n минус 1.

Ячейка 3. t по формуле и решение словами

```
se = stats.sem(experience)  # стандартная ошибка среднего
print(round(se, 3), round((experience.mean() - 15) / se, 2))  # t по формуле
'отвергаем' if result.pvalue < 0.05 else 'не отвергаем'  # решение при 0,05
```

– `stats.sem(experience)` — стандартная ошибка среднего: стандартное отклонение, делённое на корень из числа значений.

– Строка 2 — t по формуле: разница среднего выборки и 15, делённая на стандартную ошибку.

– Строка 3 — решение словами, условным выражением, как на семинаре 11.

Ячейка 4. Тест по столбцу с пропусками: ловушка без ошибки

```
gaps = stats.ttest_1samp(drivers['experience_clean'], 15)  # тест по ↩
↪ столбцу с пропусками
print(gaps)
'отвергаем' if gaps.pvalue < 0.05 else 'не отвергаем'
```

– Тест по столбцу таблицы как есть, с пропусками: все поля результата `nan`.

– Строка 3 — сравнение `nan < 0.05` даёт `False`, и решение молча становится «не отвергаем».

2 Доверительный интервал

Ячейка 5. 95-процентный интервал для среднего

```
low, high = stats.t.interval(0.95, len(experience) - 1, ↩
↪ loc=experience.mean(), scale=se)  # 95-процентный интервал
print(round(low, 2), round(high, 2))
```

– `stats.t.interval(0.95, len(experience) - 1, loc=..., scale=...)` — интервал по распределению Стьюдента. Первый аргумент — уровень доверия; второй — число степеней свободы, n минус 1; `loc` — центр, среднее выборки; `scale` — масштаб, стандартная ошибка.

– Две границы раскладываются в две переменные `low` и `high`.

Ячейка 6. Интервал по формуле руками

```
t_star = stats.t.ppf(0.975, len(experience) - 1)  # критическое значение t
half = t_star * experience.std() / np.sqrt(len(experience))  # половина ширины интервала
print(round(t_star, 3), round(experience.mean() - half, 2), round(experience.mean() + half, 2))
print(round(stats.t.ppf(0.975, 9), 2))  # t* для десяти значений
```

– `stats.t.ppf(0.975, ...)` — квантиль распределения Стьюдента: значение, ниже которого лежат 97.5 процента. На 95 процентов посередине остаётся по 2.5 процента в каждом хвосте, поэтому 0.975.

– Строка 2 — половина ширины интервала: критическое значение, умноженное на стандартную ошибку.

– Строка 4 — то же критическое значение для десяти значений, девяти степеней свободы: оно больше, и интервал шире.

Ячейка 7. Тесты против значений по обе стороны границ

```
for mu in [15.7, 15.8, 16.1, 16.2]:
    p = stats.ttest_1samp(experience, mu).pvalue  # тест против mu
    print(mu, round(p, 3), 'отвергаем' if p < 0.05 else 'не отвергаем')
```

– `for mu in [...]:` — цикл: две строки с отступом выполняются для каждого числа списка по очереди, оно попадает в переменную `mu`.

– `stats.ttest_1samp(experience, mu).pvalue` — поле результата можно взять сразу после вызова, через точку.

Ячейка 8. Интервал одного района рядом с интервалом области

```
district = drivers.loc[drivers['district'] == 'Жуковский район', 'experience_clean'].dropna()  # стаж водителей района
d_low, d_high = stats.t.interval(0.95, len(district) - 1, loc=district.mean(), scale=stats.sem(district))
print(len(district), round(district.mean(), 2), round(d_low, 2), round(d_high, 2))
print(round(high - low, 2), round(d_high - d_low, 2))  # ширины двух интервалов
print(round((d_high - d_low) / (high - low), 2), round(np.sqrt(len(experience) / len(district)), 2))  # отношение ширин и корень
```

– Строка 1 — стаж водителей Жуковского района без пропусков; переменная `district` нужна и в разделе 3.

– Строка 2 — интервал для района, как в ячейке 5; стандартная ошибка посчитана прямо в аргументе.

– Строки 4–5 — ширины двух интервалов, во сколько раз один шире другого и корень из отношения числа водителей.

3 Район как выборка

Ячейка 9. Доля района среди водителей области

```
shares = drivers.loc[drivers['experience_clean'].notna(), 'district'].value_counts(normalize=True)  # доли районов
print(round(shares['Жуковский район'] * 100, 1), round(shares['Брянский район'] * 100, 1))
```

– `drivers['experience_clean'].notna()` — маска: стаж известен. `notna` — противоположность `isna`.

– `value_counts(normalize=True)` — не числа, а доли каждого района.

– `shares['Жуковский район']` — значение по метке индекса; `* 100` — в процентах.

Ячейка 10. Двусторонняя и односторонняя альтернативы

```
two_sided = stats.ttest_1samp(district, experience.mean())  # двусторонняя альтернатива
less = stats.ttest_1samp(district, experience.mean(), alternative='less')  # среднее района меньше
print(round(two_sided.statistic, 2), round(two_sided.pvalue, 3), round(less.pvalue, 3))
```

– `stats.ttest_1samp(district, experience.mean())` — стаж района против среднего области, переданного числом.

– `alternative='less'` — односторонняя альтернатива: среднее района меньше. Значения параметра в scipy: `'two-sided'`, по умолчанию, `'less'` и `'greater'`.

Ячейка 11. Альтернатива в другую сторону: ловушка без ошибки

```
greater = stats.ttest_1samp(district, experience.mean(), alternative='greater')  # среднее района больше
print(round(greater.pvalue, 3))
```

– `alternative='greater'` — среднее района больше. Среднее района ниже областного, и p получается близким к единице: 0.952.

4 Медиана: знаковый тест

Ячейка 12. Знаковый тест против 15 и 13

```
for m0 in [15, 13]:
    above, n_signs = int((experience > m0).sum()), int((experience != m0).sum())  # выше m0; всего без равных
    print(m0, above, n_signs - above, stats.binomtest(above, n_signs).pvalue)  # знаковый тест
```

– `(experience > m0).sum()` — сколько значений выше m0; `(experience != m0).sum()` — сколько не равны m0: равные отбрасываются.

– `int(...)` — превращает число numpy в обычное целое число Python: `binomtest` принимает только такие.

– `stats.binomtest(above, n_signs)` — биномиальный тест: могло ли из `n_signs` бросков монеты выпасть `above` орлов, если монета честная. Это и есть знаковый тест медианы.

– `n_signs - above` — сколько значений ниже m0.

Ячейка 13. Тест Вилкоксона против медианы выборки: ловушка без ошибки

```
stats.wilcoxon(experience - 13).pvalue  # Вилкоксон против медианы
```

– `experience - 13` — отклонения от 13. `stats.wilcoxon(...)` — знаковый ранговый тест Вилкоксона: его нулевая гипотеза — отклонения симметричны вокруг нуля, а не медиана равна 13.

5 Доля

Ячейка 14. z-тест для доли ДТП с погибшими

```
dead = dtp['dead_count'] > 0  # ДТП с погибшими
count, nobs = dead.sum(), len(dtp)
z, p = proportions_ztest(count, nobs, 0.15, prop_var=0.15)  # z-тест для доли
print(count, nobs, round(count / nobs, 4), round(z, 2), p)
```

– Строка 2 — сколько ДТП с погибшими и сколько ДТП всего.

– `proportions_ztest(count, nobs, 0.15, prop_var=0.15)` — z-тест для доли. Аргументы по порядку: сколько «да», сколько всего, проверяемая доля. `prop_var=0.15` — стандартная ошибка считается по проверяемой доле; так тест согласован с интервалом Вилсона.

– Результат — два числа в скобках, z и p, без имён полей; они раскладываются в две переменные.

Ячейка 15. Интервал Вилсона

```
low_share, high_share = proportion_confint(count, nobs, method='wilson')  ↩
↪ # интервал Вилсона
print(round(low_share, 4), round(high_share, 4))
print(round(proportions_ztest(count, nobs, 0.14, prop_var=0.14)[1], 3))  # ↩
↪ тест против 0,14
```

– `proportion_confint(count, nobs, method='wilson')` — 95-процентный интервал для доли методом Вилсона. Без `method` функция строит интервал по нормальному приближению, а он ошибается на малых выборках.

– `proportions_ztest(...)[1]` — второе из двух чисел результата, p: счёт с нуля.

Ячейка 16. Тест без prop_var на 40 ДТП: ловушка без ошибки

```
print([round(b, 3) for b in proportion_confint(10, 40, method='wilson')])  # интервал Вилсона
for p0 in [0.40, 0.14]:
    wald = proportions_ztest(10, 40, p0)[1]  # ошибка по доле выборки
    score = proportions_ztest(10, 40, p0, prop_var=p0)[1]  # ошибка по p0
    print(p0, round(wald, 3), round(score, 3))
```

– `[round(b, 3) for b in ...]` — список, собранный циклом в одну строку: каждая граница интервала округлена.

– В цикле — два теста для каждой доли: без `prop_var`, это тест Вальда, ошибка по доле выборки, и с `prop_var`, ошибка по проверяемой доле.

Ячейка 17. Перепутанные аргументы: ловушка без ошибки

```
proportions_ztest(nobs, count, 0.15, prop_var=0.15)  # аргументы перепутаны
```

– Сначала «всего», потом «сколько»: функция не проверяет порядок, и «доля» выходит больше единицы, z — 752.9. Надёжнее по имени: `proportions_ztest(count=count, nobs=nobs, value=0.15, prop_var=0.15)`.

Ячейка 18. Доля ДТП по выходным

```
weekend = dtp['datetime'].dt.day_of_week >= 5  # суббота и воскресенье
z_w, p_w = proportions_ztest(weekend.sum(), len(dtp), 2 / 7, prop_var=2 / 7)  # против двух седьмых
print(weekend.sum(), round(weekend.mean(), 4), round(2 / 7, 4), round(z_w, 2), round(p_w, 3))
```

– `dt.day_of_week >= 5` — маска выходных: суббота — 5, воскресенье — 6.

– Строка 2 — тот же z-тест, проверяемая доля — две седьмых. Имена `z_w` и `p_w` другие, чтобы не затереть `z` и `p` ячейки 14: они нужны в таблице раздела 7.

Ячейка 19. Обычный интервал и интервал Вилсона рядом

```
small = dtp[dtp['district'] == 'Рогнединский район']  # самый маленький район
for k, n in [(count, nobs), ((small['dead_count'] > 0).sum(), len(small))]:
    normal = proportion_confint(k, n)  # нормальное приближение
    wilson = proportion_confint(k, n, method='wilson')  # интервал Вилсона
    print(n, k, [round(b, 3) for b in normal], [round(b, 3) for b in wilson])
```

– `for k, n in [(...), (...)]:` — цикл по списку из двух пар чисел: в каждом шаге пара раскладывается в `k` и `n`. Первая пара — вся область, вторая — маленький район.

– `proportion_confint(k, n)` — интервал по нормальному приближению, по умолчанию; рядом — Вилсон.

6 Согласие с распределением

Ячейка 20. ДТП по дням недели

```
counts = dtp['datetime'].dt.day_of_week.value_counts().sort_index()  # ДТП по дням недели
counts.tolist()
```

– `value_counts().sort_index()` — число ДТП по номерам дней, по порядку с понедельника.

– `.tolist()` — значения обычным списком Python: так их короче напечатать.

Ячейка 21. Ожидаемые долями: намеренная ошибка

```
stats.chisquare(counts, [1 / 7] * 7)  # ожидаемые долями
```

– `[1 / 7] * 7` — список из семи одинаковых чисел: умножение списка на число повторяет его.

– `stats.chisquare(наблюдаемые, ожидаемые)` — хи-квадрат согласия. Сумма наблюдаемых — 10 114, сумма ожидаемых — единица, и scipy отказывается: `ValueError`. Ошибка намеренная.

Ячейка 22. Ожидаемые в числах ДТП и тест

```
expected = pd.Series(counts.sum() / 7, index=counts.index)  # ожидаемые ДТП в день
chi = stats.chisquare(counts, expected)  # хи-квадрат согласия
print(round(expected.iloc[0], 1), round(chi.statistic, 1), chi.pvalue)
```

– `pd.Series(counts.sum() / 7, index=counts.index)` — Series, в котором одно и то же число — все ДТП, делённые на 7, — повторено для каждого дня.

– `chi = stats.chisquare(counts, expected)` — тест; результат с полями `statistic` и `pvalue`.

– `expected.iloc[0]` — первое значение по позиции: ожидаемое на один день.

Ячейка 23. Вклад каждого дня и столбцы с линией ожидаемого

```
contribution = (counts - expected) ** 2 / expected  # вклад каждого дня
days = ['пн', 'вт', 'ср', 'чт', 'пт', 'сб', 'вс']
fig, ax = plt.subplots(figsize=(8, 4))
ax.bar(days, counts)  # наблюдаемые ДТП
ax.axhline(expected.iloc[0], color='red')  # ожидаемые при равенстве
ax.set_title('ДТП по дням недели и равномерное распределение')
ax.set_xlabel('День недели')
ax.set_ylabel('Число ДТП')
contribution.round(1).tolist()
```

– `(counts - expected) ** 2 / expected` — слагаемое статистики для каждого дня: квадрат разницы, делённый на ожидаемое. `**` — возведение в степень.

– `ax.bar(days, counts)` — столбцы matplotlib: подписи по оси X и высоты.

– `ax.axhline(..., color='red')` — горизонтальная линия через всю рамку на высоте ожидаемого числа.

7 Много проверок

Ячейка 24. Районы против доли по районам, с поправкой

```
1 city = dtp['district'] == 'Брянская область'  # ДТП без района: Брянск
2 print(round(dead[city].mean(), 3), round(dead[~city].mean(), 3))  # доли с ↩
  ↪ погибшими
3 share_districts = dead[~city].mean()  # доля по районам
4 by_district = ↩
  ↪ dtp[~city].assign(dead=dead).groupby('district')['dead'].agg(['sum', ↩
  ↪ 'count'])  # ДТП с погибшими по районам
5 by_district = by_district[by_district['count'] >= 50]  # районы от 50 ДТП
6 by_district['p'] = [proportions_ztest(k, n, share_districts, ↩
  ↪ prop_var=share_districts)[1] for k, n in zip(by_district['sum'], ↩
  ↪ by_district['count'])]
7 print(len(by_district), (by_district['p'] < 0.05).sum(), round(0.05 / ↩
  ↪ len(by_district), 4), (by_district['p'] < 0.05 / len(by_district)).sum())
8 by_district['доля'] = (by_district['sum'] / by_district['count']).round(3) ↩
  ↪  # доля ДТП с погибшими
9 by_district[by_district['p'] < 0.05].sort_values('p')
```

– `dead[city].mean()` — доля ДТП с погибшими в городе; `dead[~city].mean()` — в районах.

– `dtp[~city].assign(dead=dead)` — таблица без города с новым столбцом `dead`; `assign` возвращает новую таблицу, `dtp` не меняется.

– `.groupby('district')['dead'].agg(['sum', 'count'])` — по каждому району: сколько ДТП с погибшими и сколько ДТП всего.

– Строка 6 — p для каждого района списком: `zip` идёт по двум столбцам сразу и даёт пары «с погибшими, всего», по одной на район.

– Строка 7 — число районов, сколько из них значимы при 0.05, порог с поправкой Бонферрони, `0.05 / len(by_district)`, и сколько значимы с поправкой.

– Последняя строка — районы с p меньше 0.05, по возрастанию p.

8 Этап 3: сводная таблица гипотез

Ячейка 25. Таблица четырёх гипотез занятия

```
report = pd.DataFrame([
    {'вопрос': 'средний стаж 15 лет?', 'тест': 't-тест', 'n': len(experience), 'статистика': result.statistic, 'p': result.pvalue},
    {'вопрос': 'стаж в Жуковском районе как в области?', 'тест': 't-тест', 'n': len(district), 'статистика': two_sided.statistic, 'p': two_sided.pvalue},
    {'вопрос': 'доля ДТП с погибшими 0,15?', 'тест': 'z-тест доли', 'n': nobs, 'статистика': z, 'p': p},
    {'вопрос': 'ДТП по дням недели поровну?', 'тест': 'хи-квадрат', 'n': counts.sum(), 'статистика': chi.statistic, 'p': chi.pvalue},
], index=range(1, 5))
report['решение'] = np.where(report['p'] < 0.05, 'отвергаем', 'не отвергаем')  # решение при 0,05
report.round(3)
```

– `pd.DataFrame([{...}, {...}])` — таблица из списка словарей: каждый словарь — строка, ключи — имена столбцов. Числа берутся из переменных ранних ячеек: `result`, `two_sided`, `z`, `p`, `chi`.

– `index=range(1, 5)` — строки пронумерованы с единицы: 1, 2, 3, 4.

– `np.where(...)` — решение словами по каждой строке, как на семинаре 11.